Name:	Richard Benson Raj.B.A
Course:	Artificial Intelligence
Reg no:	1925260077
Course code:	CSA1711
Slot:	D

13. Search-and-Rescue Robot in a Collapsed Building (LRTA* and Online Search)

i. Online search problem definition

- **State:** the robot's current location (cell/junction) plus what it has sensed locally (nearby walls, openings, estimated distance to survivor signal). The map is unknown and is built as the robot moves.
- **ACTIONS(s):** the moves available in the current state, discovered only by sensing: Forward, Back, Left, Right through detected openings (and Stop/Wait). The robot learns the actions only after it reaches the state.
- **Step cost c(s,a,s'):** the effort of one move, e.g. 1 per cell, or distance/energy/time travelled, with an added penalty for risky or unstable passages. Cost is known only after the move is made.
- **Goal test:** GOAL(s) is true when the robot is at the survivor's location (signal strength at maximum / survivor confirmed within sensor range).
- **Heuristic h(s):** estimated distance to the survivor from the signal strength (e.g. straight-line distance). It is rough and may be inaccurate, and it is stored as H(s) in a table that is improved during exploration.
- **Special property:** the environment is **not safely explorable** because some passages collapse behind the robot and become one-way (irreversible actions).

ii. Competitive ratio and why dead ends are dangerous

Competitive ratio = (cost the online agent actually travels) / (cost of the optimal path if the agent had known the whole map in advance). A ratio of 1 is perfect; a larger ratio means more wasted effort caused by not knowing the environment. It measures how well an online algorithm performs compared with an offline one.

Why dead ends are dangerous:

- An action into a one-way passage **cannot be undone**; the robot cannot backtrack to try another route.
- If the robot enters a dead end (or a passage that collapses behind it), it may become permanently trapped, so it can never reach the survivor.
- In that case the competitive ratio becomes **infinite** (the optimal offline agent would have avoided the trap, but the online agent cannot).
- An adversary (the environment) can always place the trap where the agent chooses to go, so no online algorithm can guarantee a bounded ratio in a general environment with irreversible actions. Hence safety (avoiding irreversible moves) matters more than speed.

iii. Online depth-first search and need for reversible actions

- The agent keeps a table **result[s,a]** of the outcome of each action tried, a set of visited states, and a list of **unbacktracked** states for each state.
- At the current state it picks an **untried action** and moves to a new state, going deeper and deeper (like DFS), recording the result.
- When all actions in a state are tried (a dead end in the explored sense), it **physically backtracks** to the most recent predecessor in the unbacktracked list and tries its remaining actions.
- The search ends when the goal (survivor) is reached, or when no unexplored actions remain.
- **Why reversible actions are required:** backtracking means doing the inverse of earlier moves. If an action is irreversible (passage collapses and becomes one-way), the robot cannot return, so the DFS may get stuck and lose completeness. Online DFS is only guaranteed to work in **safely explorable**, reversible (undirected-graph) state spaces. Its worst-case cost is about 2 x (number of edges) moves.

iv. LRTA* algorithm

LRTA* (Learning Real-Time A*) is an online agent that combines **local, one-step lookahead** with **learning** of the heuristic. It keeps a table H(s), initialised with h(s) and updated as the robot moves.

Steps:

- At current state s, for each available action a compute **cost estimate** LRTA*-COST(s,a,s',H) = c(s,a,s') + H(s') (use h(s') if s' is not yet in the table; unexplored actions are treated optimistically with cost h(s)).
- **Update:** H(s) = min over a of [c(s,a,s') + H(s')]. This raises H(s) to a more accurate value based on the neighbours.
- **Move:** choose the action a that gives the minimum value and go to s'. Repeat until the goal is reached.

Escaping local minima: a pure greedy/hill-climbing agent gets stuck in a state whose neighbours all look worse (a local minimum of h). In LRTA*, each time the robot stays in such a region, the update raises H(s) of those states (because H(s) = c + H(s') becomes larger than the old estimate). As the values rise, the robot's current region looks less attractive and it is pushed out toward other states with lower estimates. Thus the robot fills the 'bowl' of the local minimum and leaves it. Since h is admissible (never overestimates), LRTA* is complete in finite, **safely explorable** spaces.

Example: robot at A (H=3) with neighbours B (H=4, cost 1) and C (H=5, cost 1). Update: H(A) = min(1+4, 1+5) = 5. It moves to B. Now A's estimate is 5, so if B leads back to A the robot will not keep oscillating, as A is now less attractive.

v. Recommended approach and justification

Recommendation: use **LRTA*** with the survivor-signal distance as the heuristic, combined with a **safety layer**: (a) assign a high cost / large penalty to unstable or possibly one-way passages, (b) prefer reversible moves and explore unstable passages only when no safe alternative exists, (c) keep a record of visited cells, and (d) optionally drop communication relays or markers so the robot can return.

- **Safety:** online DFS depends on backtracking, which fails if passages collapse. LRTA* only needs local information and does not assume a map, and with penalty costs it avoids risky passages. Its learned H values prevent repeated visits and loops.
- **Speed:** the signal-based heuristic guides the robot toward the survivor, and H updates let it quickly escape local minima (e.g. a wall between robot and survivor), giving much faster search than blind DFS.

- **Real-time:** each decision takes constant time, suitable for a small robot with limited computing power, and the learned H table improves over repeated trials.
- **Limitation:** no algorithm can guarantee success against irreversible dead ends, so the safety rules above are essential.

15. Airline Gate Assignment (Simulated Annealing)

i. Optimization problem formulation

- **State:** a complete assignment of all 120 flights to the 30 gates, e.g. a vector G = (g1, g2, ..., g120) where gi is the gate (1..30) of flight i (with its time slot).
- **Neighbour moves:** (1) move one flight to a different gate; (2) swap the gates of two flights; (3) shift a flight to another feasible gate in a nearby time. Moves that break aircraft-size limits are rejected or penalised.
- **Objective function (to minimise):** E(G) = w1 x (total passenger walking distance) + w2 x (number of gate conflicts / overlapping flights at the same gate) + w3 x (aircraft size violations, treated as a very large penalty). Walking distance = sum over flights of passengers x distance from gate to terminal/transfer point.
- **Constraints:** no two flights occupy the same gate at the same time (with turnaround buffer); aircraft size must be at most the gate's limit.

ii. Why hill climbing gets stuck

- Hill climbing accepts only moves that improve the objective, so it stops at the first state where no single move/swap gives a better value, a **local optimum**.
- The landscape has many **local optima, plateaus and ridges**: fixing one conflict by moving a flight can create another conflict or increase walking distance, so a single move looks worse even if a combination of moves would be much better.
- Hard constraints (gate size, conflicts) create barriers between good regions; hill climbing cannot cross them because it never accepts a worse state.
- The result depends heavily on the random starting schedule, giving poor schedules.

iii. Simulated annealing and acceptance probability

Simulated annealing (SA) is inspired by annealing in metallurgy. At each step it picks a **random neighbour**. If the neighbour is better it is always accepted. If it is worse it is accepted with probability **P = e^(ΔE/T)**, where ΔE = (value of new state) - (value of current state) in a maximisation form (ΔE is negative for a worse move), and T is the temperature. (For a cost minimisation form this is e^(-Δcost/T).) At high T, bad moves are often accepted (exploration); as T falls, SA behaves like hill climbing (exploitation).

Numerical example: current total walking distance (cost) = 5000 m. A neighbour move gives 5100 m, a worse move by 100, so ΔE = -100.

Temperature T	P = e^(-100/T)	Meaning
T = 500	e^(-0.2) = 0.82	Accepted about 82% of the time (high exploration)
T = 100	e^(-1) = 0.37	Accepted about 37% of the time
T = 20	e^(-5) = 0.0067	Accepted about 0.7% of the time (almost hill climbing)

A random number r in [0,1] is drawn; if r < P the worse move is accepted. If the neighbour had been better (e.g. 4900 m) it would be accepted with probability 1.

iv. Temperature schedule

- The schedule defines how T decreases over time, e.g. geometric cooling T(k+1) = α x T(k), with α about 0.95 to 0.99, starting at a high T0 and stopping at a small T_min.

- **High T:** broad exploration; escapes local optima. **Low T:** fine-tuning around a good solution.
- **If T is lowered too quickly** (rapid cooling/quenching): the probability of accepting worse moves drops to near zero early, so SA behaves like hill climbing, gets trapped in a poor local optimum and loses its guarantee of finding the global optimum. The slower the cooling, the better the quality (the global optimum is found with probability approaching 1 if cooling is sufficiently slow), at the cost of more running time.

v. Simulated annealing vs genetic algorithm, and recommendation

Aspect	Simulated Annealing	Genetic Algorithm
Search style	Single solution improved step by step	Population of solutions evolved by selection, crossover, mutation
Escaping local optima	Accepting worse moves with probability e^(ΔE/T)	Diversity from population, crossover and mutation
Implementation	Simple; only neighbour move + cost function + schedule	More complex: encoding, crossover that preserves feasibility, parameters
Constraints	Easy to handle with move design and penalties	Crossover may produce invalid schedules needing repair
Memory/time per step	Low memory, cheap steps	Higher memory and computation per generation
Parallelism / diversity	Limited (single path)	Naturally parallel, explores many regions

Recommendation: Simulated annealing. The gate-assignment problem has a natural neighbourhood (move/swap flights), strict constraints that are easy to keep feasible with small moves, and a moderate size (120 flights, 30 gates). SA is simpler, uses little memory, and directly fixes the hill-climbing weakness. With a well-chosen cooling schedule (and a few restarts) it gives high-quality schedules. A genetic algorithm could be considered for larger networks or when many alternative schedules are wanted, but its crossover often breaks feasibility and needs more tuning.